O'Reilly's CD bookshelfs | FreeBSD | Linux | Cisco | e-Reading Library G+1 2

Chapter 3. Remote Method Invocation

Contents:

Introduction to RMI
Defining Remote Objects
Creating the Stubs and Skeletons
Accessing Remote Objects as a Client
Dynamically Loaded Classes
Remote Object Activation
RMI and Native Method Calls
RMI over IIOP

This chapter examines the Java Remote Method Invocation (RMI) API--Java's native scheme for creating and using remote objects. Java RMI provides the following elements:

- Remote object implementations
- Client interfaces, or stubs, to remote objects
- A remote object registry for finding objects on the network
- A network protocol for communication between remote objects and their client
- A facility for automatically creating (activating) remote objects on-demand



Each of these elements (except the last one) has a Java interface defined for it within the `java.rmi` package and its subpackages, which comprise the RMI API. Using these interfaces, you can develop remote objects and the clients that use them to create a distributed application that resides on hosts across the network.

3.1. Introduction to RMI

RMI is the distributed object system that is built into the core Java environment. You can think of RMI as a built-in facility for Java that allows you to interact with objects that are actually running in Java virtual machines on remote hosts on the network. With RMI (and other distributed object APIs we discuss in this book), you can get a reference to an object that "lives" in a remote process and invoke methods on it as if it were a local object running within the same virtual machine as your code (hence the name, "Remote Method Invocation API").

RMI was added to the core Java API in Version 1.1 of the JDK (and enhanced for Version 1.2 of the Java 2 platform), in recognition of the critical need for support for distributed objects in distributed-application development. Prior to RMI, writing a distributed application involved basic socket programming, where a "raw" communication channel was used to pass messages and data between two remote processes. Now, with RMI and distributed objects, you can "export" an object as a remote object, so that other remote processes/agents can access it directly as a Java object. So, instead of defining a low-level message protocol and data transmission format between processes in your distributed application, you use Java interfaces as the "protocol" and the exported method arguments become the data transmission format. The distributed object system (RMI in this case) handles all the underlying networking needed to make your remote method calls work.

Java RMI is a Java-only distributed object scheme; the objects in an RMI-based distributed application have to be implemented in Java. Some other distributed object schemes, most notably CORBA, are language-independent, which means that the objects can be implemented in any language that has a defined binding. With CORBA, for example, bindings exist for C, C++, Java, Smalltalk, and Ada, among other languages.

The advantages of RMI primarily revolve around the fact that it is "Java-native." Since RMI is part of the core Java API and is built to work directly with Java objects within the Java VM, the integration of its remote object facilities into a Java application is almost seamless. You really can use RMI-enabled objects as if they live in the local Java environment. And since Java RMI is built on the assumption that both the client and server are Java objects, RMI can extend the internal garbage-collection mechanisms of the standard Java VM to provide distributed garbage collection of remotely exported objects.

If you have a distributed application with heterogeneous components, some of which are written in Java and

some that aren't, you have a few choices. You can use RMI, wrapping the non-Java code with RMI-enabled Java objects using the Java Native Interface (JNI). At the end of this chapter, we discuss this first option in some detail, to give you a feeling for where it could be useful and where it wouldn't. Another option is to use another object distribution scheme, such as CORBA, that supports language-independent object interfaces. Chapter 4, "Java IDL", covers the Java interface to CORBA that is included in the Java 2 SDK. A third option involves the new RMI/IIOP functionality that allows RMI objects to communicate directly with remote CORBA objects over IIOP. We also discuss this option in some detail at the end of this chapter.

3.1.1. RMI in Action

Before we start examining the details of using RMI, let's look at a simple RMI remote object at work. We can create an `Account` object that represents some kind of bank account and then use RMI to export it as a remote object so that remote clients (e.g., ATMs, personal finance software running on a PC) can access it and carry out transactions.

The first step is to define the interface for our remote object. Example 3-1 shows the `Account` interface. You can tell that it's an RMI object because it extends the `java.rmi.Remote` interface. Another signal that this is meant for remote access is that each method can throw a `java.rmi.RemoteException`. The `Account` interface includes methods to get the account name and balance and to make deposits, withdrawals, and transfers.

Example 3-1. A Remote Account Interface

```
import java.rmi.Remote;
import java.rmi.RemoteException;
import java.util.List;

public interface Account extends Remote {
  public String getName() throws RemoteException;
  public float getBalance() throws RemoteException;
  public void withdraw(float amt) throws RemoteException;
  public void deposit(float amt) throws RemoteException;
  public void transfer(float amt, Account src) throws RemoteException;
  public void transfer(List amts, List srcs) throws RemoteException;
}
```

The next step is to create an implementation of this interface, which leads to the `AccountImpl` class shown in Example 3-2. This class implements all the methods listed in the `Account` interface and adds a constructor that takes the name of the new account to be created. Notice that the `AccountImpl` class implements the

`Account` interface, but it also extends the `java.rmi.UnicastRemoteObject` class. This RMI class provides some of the basic remote functionality for server objects.

Example 3-2. Implementation of the Remote Account Interface

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;
import java.util.List;
import java.util.ListIterator;

public class AccountImpl extends UnicastRemoteObject implements Account {
  private float mBalance = 0;
  private String mName = "";

  // Create a new account with the given name
  public AccountImpl(String name) throws RemoteException {
    mName = name;
  }

  public String getName() throws RemoteException {
    return mName;
  }
  public float getBalance() throws RemoteException {
    return mBalance;
  }

  // Withdraw some funds
  public void withdraw(float amt) throws RemoteException {
    mBalance -= amt;
    // Make sure balance never drops below zero
    mBalance = Math.max(mBalance, 0);
  }

  // Deposit some funds
  public void deposit(float amt) throws RemoteException {
    mBalance += amt;
  }

  // Move some funds from another (remote) account into this one
  public void transfer(float amt, Account src) throws RemoteException {
    src.withdraw(amt);
    this.deposit(amt);
```

```
  }

  // Make several transfers from other (remote) accounts into this one
  public void transfer(List amts, List srcs) throws RemoteException {
    ListIterator amtCurs = amts.listIterator();
    ListIterator srcCurs = srcs.listIterator();
    // Iterate through the accounts and the amounts to be transferred from
    // each (assumes amounts are given as Float objects)
    while (amtCurs.hasNext() && srcCurs.hasNext()) {
      Float amt = (Float)amtCurs.next();
      Account src = (Account)srcCurs.next();
      this.transfer(amt.floatValue(), src);
    }
  }
}
```

Once the remote interface and an implementation of it are complete, you need to compile both Java files with your favorite Java compiler. After this is done, you use the RMI stub/skeleton compiler to generate a client stub and a server skeleton for the `AccountImpl` object. The stub and skeleton handle the communication between the client application and the server object. With Sun's Java SDK, the RMI compiler is called *rmic*, and you can invoke it for this example like so:

```
% rmic -d /home/classes AccountImpl
```

The stub and skeleton classes are generated and stored in the directory given by the `-d` option (*/home/classes*, in this case). This example assumes that the `AccountImpl` class is already in your `CLASSPATH` before you run the RMI compiler.

There's just one more thing we need to do before we can actually use our remote object: register it with an RMI registry, so that remote clients can find it on the network. The utility class that follows, `RegAccount`, does this by creating an `AccountImpl` object and then binding it to a name in the local registry using the `java.rmi.Naming` interface. After it's done registering the object, the class goes into a `wait()`, which allows remote clients to connect to the remote object:

```
import java.rmi.Naming;

public class RegAccount {
  public static void main(String argv[]) {
    try {
```

```
      // Make an Account with a given name
      AccountImpl acct = new AccountImpl("JimF");

      // Register it with the local naming registry
      Naming.rebind("JimF", acct);
      System.out.println("Registered account.");
    }
    catch (Exception e) {
      e.printStackTrace();


    }
  }
}
```

After you compile the `RegAccount` class, you can run its `main()` method to register an `Account` with the local RMI registry. First, however, you need to start the registry. With Sun's Java SDK, the registry can be started using the *rmiregistry* utility. On a Unix machine, this can be done like so:

```
objhost% rmiregistry &
```

Once the registry is started, you can invoke the `main()` method on the `RegAccount` class simply by running it:

```
objhost% java RegAccount
Registered account.
```

Now we have a remote `Account` object that is ready and waiting for a client to access it and call its methods. The following client code does just this, by first looking up the remote `Account` object using the `java.rmi.Naming` interface (and assuming that the `Account` object was registered on a machine named *objhost.org*), and then calling the deposit method on the `Account` object:

```
import java.rmi.Naming;

public class AccountClient {
  public static void main(String argv[]) {
    try {
      // Lookup account object
      Account jimAcct = (Account)Naming.lookup("rmi://objhost.org/JimF");
```

```
      // Make deposit
      jimAcct.deposit(12000);

      // Report results and balance.
      System.out.println("Deposited 12,000 into account owned by " +
                         jimAcct.getName());
      System.out.println("Balance now totals: " + jimAcct.getBalance());
    }
    catch (Exception e) {
      System.out.println("Error while looking up account:");
      e.printStackTrace();
    }
  }
}
```

The first time you run this client, here's what you'd do:

```
% java AccountClient
Deposited 12,000 into account owned by JimF
Balance now totals: 12000.0
```

For the sake of this example, I've assumed that the client process is running on a machine with all the necessary classes available locally (the `Account` interface and the stub and skeleton classes generated from the `AccountImpl` implementation). Later in the chapter, we'll see how to deal with loading these classes remotely when the client doesn't have them locally.

3.1.2. RMI Architecture

Now that we've seen a complete example of an RMI object in action, let's look at what makes remote objects work, starting with an overview of the underlying RMI architecture. There are three layers that comprise the basic remote-object communication facilities in RMI:

- The *stub/skeleton* layer, which provides the interface that client and server application objects use to interact with each other.
- The *remote reference* layer, which is the middleware between the stub/skeleton layer and the underlying transport protocol. This layer handles the creation and management of remote object references.



- The *transport protocol* layer, which is the binary data protocol that sends remote object requests over the wire.

These layers interact with each other as shown in Figure 3-1. In this figure, the server is the application that provides remotely accessible objects, while the client is any remote application that communicates with these server objects.

In a distributed object system, the distinctions between clients and servers can get pretty blurry at times. Consider the case where one process registers a remote-enabled object with the RMI naming service, and a number of remote processes are accessing it. We might be tempted to call the first process the server and the other processes the clients. But what if one of the clients calls a method on the remote object, passing a reference to an RMI object that's local to the client. Now the server has a reference to and is using an object exported from the client, which turns the tables somewhat. The "server" is really the server for one object and the client of another object, and the "client" is a client and a server, too. For the sake of discussion, I'll refer to a process in a distributed application as a server or client if its role in the overall system is generally limited to one or the other. In peer-to-peer systems, where there is no clear client or server, I'll refer to elements of the system in terms of application-specific roles (e.g., chat participant, chat facilitator).

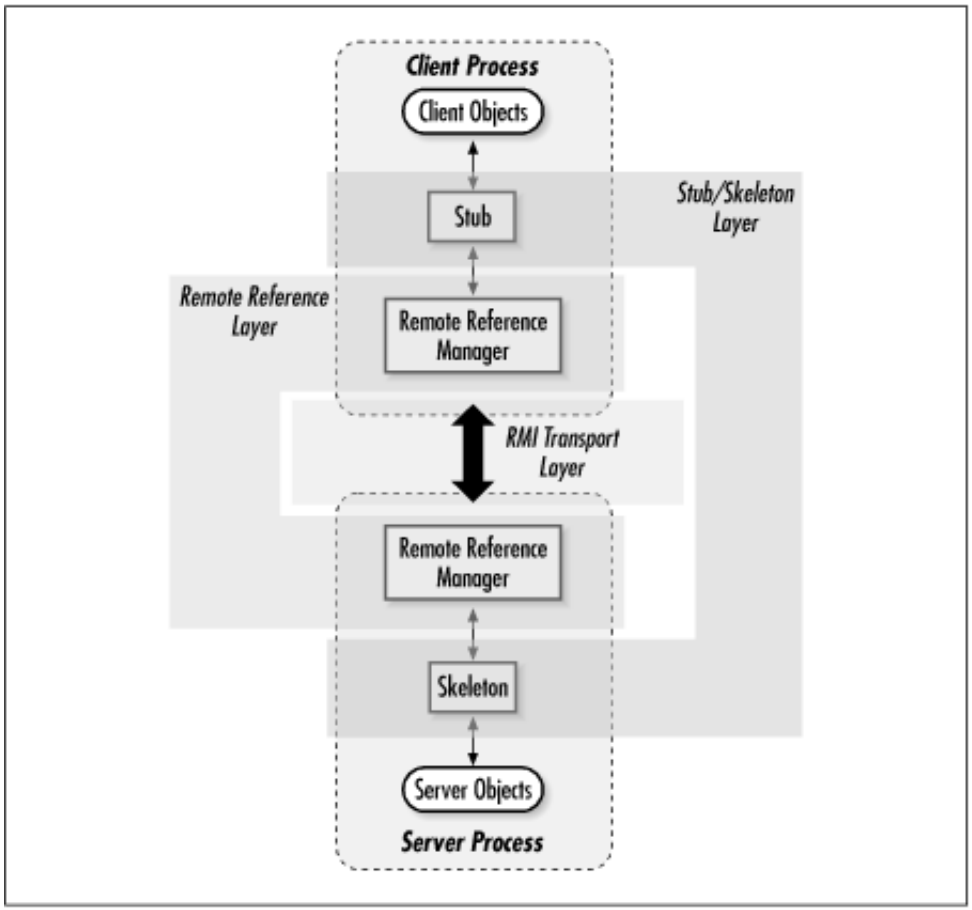


Figure 3-1. The RMI runtime architecture

As you can see in Figure 3-1, a client makes a request of a remote object using a client-side stub; the server object receives this request from a server-side object skeleton. A client initiates a remote method invocation by calling a method on a stub object. The stub maintains an internal reference to the remote object it represents and forwards the method invocation request through the remote reference layer by *marshalling* the method arguments into serialized form and asking the remote reference layer to forward the method request and arguments to the appropriate remote object. Marshalling involves converting local objects into portable form so that they can be transmitted to a remote process. Each object is checked as it is marshaled, to determine whether it implements the `java.rmi.Remote` interface. If it does, its remote reference is used as its marshaled data. If it isn't a `Remote` object, the argument is serialized into bytes that are sent to the remote host and reconstituted into a copy of the local object. If the argument is neither `Remote` nor `Serializable`, the stub

throws a `java.rmi.MarshalException` back to the client.

If the marshalling of method arguments succeeds, the client-side remote reference layer receives the remote reference and marshaled arguments from the stub. This layer converts the client request into low-level RMI transport requests according to the type of remote object communication being used. In RMI, remote objects can (potentially) run under several different communication styles, such as point-to-point object references, replicated objects, or multicast objects. The remote reference layer is responsible for knowing which communication style is in effect for a given remote object and generating the corresponding transport-level requests. In the current version of RMI (Version 1.2 of Java 2), the only communication style provided out of the box is point-to-point object references, so this is the only style we'll discuss in this chapter. For a point-to-point communication, the remote reference layer constructs a single network-level request and sends it over the wire to the sole remote object that corresponds to the remote reference passed along with the request.

On the server, the server-side remote reference layer receives the transport-level request and converts it into a request for the server skeleton that matches the referenced object. The skeleton converts the remote request into the appropriate method call on the actual server object, which involves *unmarshalling* the method arguments into the server environment and passing them to the server object. As you might expect, unmarshalling is the inverse procedure to the marshalling process on the client. Arguments sent as remote references are converted into local stubs on the server, and arguments sent as serialized objects are converted into local copies of the originals.

If the method call generates a return value or an exception, the skeleton marshals the object for transport back to the client and forwards it through the server reference layer. This result is sent back using the appropriate transport protocol, where it passes through the client reference layer and stub, is unmarshaled by the stub, and is finally handed back to the client thread that invoked the remote method.

3.1.3. RMI Object Services

On top of its remote object architecture, RMI provides some basic object services you can use in your distributed application. These include an object naming/registry service, a remote object activation service, and distributed garbage collection.

3.1.3.1. Naming/registry service

When a server process wants to export some RMI-based service to clients, it does so by registering one or more RMI-enabled objects with its local RMI registry (represented by the `Registry` interface). Each object is registered with a name clients can use to reference it. A client can obtain a stub reference to the remote object

by asking for the object by name through the `Naming` interface. The `Naming.lookup()` method takes the fully qualified name of a remote object and locates the object on the network. The object's fully qualified name is in a URL-like syntax that includes the name of the object's host and the object's registered name.

It's important to note that, although the `Naming` interface is a default naming service provided with RMI, the RMI registry can be tied into other naming services by vendors. Sun has provided a binding to the RMI registry through the Java Naming and Directory Interface (JNDI), for example. See Chapter 6, "JNDI", for more details on how JNDI can be used to look up objects (remote or otherwise).

Once the `lookup()` method locates the object's host, it consults the RMI registry on that host and asks for the object by name. If the registry finds the object, it generates a remote reference to the object and delivers it to the client process, where it is converted into a stub reference that is returned to the caller. Once the client has a remote reference to the server object, communication between the client and the server commences as described earlier. We'll talk in more detail about the `Naming` and `Registry` interfaces later in this chapter.

3.1.3.2. Object activation service

The remote object activation service is new to RMI as of Version 1.2 of the Java 2 platform. It provides a way for server objects to be started on an as-needed basis. Without remote activation, a server object has to be registered with the RMI registry service from within a running Java virtual machine. A remote object registered this way is only available during the lifetime of the Java VM that registered it. If the server VM halts or crashes for some reason, the server object becomes unavailable and any existing client references to the object become invalid. Any further attempts by clients to call methods through these now-invalid references result in RMI exceptions being thrown back to the client.

The RMI activation service provides a way for a server object to be activated automatically when a client requests it. This involves creating the server object within a new or existing virtual machine and obtaining a reference to this newly created object for the client that caused the activation. A server object that wants to be activated automatically needs to register an activation method with the RMI activation daemon running on its host. We'll discuss the RMI activation service in more detail later in the chapter.

3.1.3.3. Distributed garbage collection

The last of the remote object services, distributed garbage collection, is a fairly automatic process that you as an application developer should never have to worry about. Every server that contains RMI-exported objects automatically maintains a list of remote references to the objects it serves. Each client that requests and receives a reference to a remote object, either explicitly through the registry/naming service or implicitly as

the result of a remote method call, is issued this remote object reference through the remote reference layer of the object's host process. The reference layer automatically keeps a record of this reference in the form of an expirable "lease" on the object. When the client is done with the reference and allows the remote stub to go out of scope, or when the lease on the object expires, the reference layer on the host automatically deletes the record of the remote reference and notifies the client's reference layer that this remote reference has expired. The concept of expirable leases, as opposed to strict on/off references, is used to deal with situations where a client-side failure or a network failure keeps the client from notifying the server that it is done with its reference to an object.

When an object has no further remote references recorded in the remote reference layer, it becomes a candidate for garbage collection. If there are also no further local references to the object (this reference list is kept by the Java VM itself as part of its normal garbage-collection algorithm), the object is marked as garbage and picked up by the next run of the system garbage collector.

PREVIOUS | HOME | NEXT

2.13. JDBC 2.0 | BOOK INDEX | 3.2. Defining Remote Objects



Copyright © 2001 O'Reilly & Associates. All rights reserved.

Monitor Java

Monitor Java App Performance with Datadog. Try Free Now!

bigmir)net 4495 7223 Участник Rambler's TOP 100 163 4409 7222 +5317 (3485) РЕЙТИНГ 60166098 mail.ru 6786 4291 UARating 62446675 7298 4520